Quantum Logic Gates and Composition

Lab Handout

Session Overview

Duration: 1h15

Session Objectives

This lab combines matrix reasoning, circuit visualization, and Qiskit codes to bridge the gap between symbolic logic and quantum computation.

- Understand classical vs quantum gate behavior.
- Visualize how unitary matrices map states on the Bloch sphere.
- Learn to decompose complex circuits into a universal gate set.

Agenda

0:00-0:10	Classical vs Quantum Logic
0:10-0:35	Single-Qubit Gates (Pauli, H, S, T)
0:35-0:55	Multi-Qubit Gates (CNOT, Toffoli, SWAP)
0:55-1:10	Universal Gates & Decomposition
1:10-1:15	Optimization & Wrap-up Exercises

Setup

```
from qiskit import QuantumCircuit
from qiskit.quantum_info import Operator, Statevector
from qiskit.visualization import plot_bloch_multivector, plot_state_qsphere
from qiskit_aer import AerSimulator
import numpy as np
import matplotlib.pyplot as plt

sim = AerSimulator()
print('Quantum gates ready!')
```

1 Classical vs Quantum Gates

Classical logic gates (AND, OR, NOT) are deterministic and irreversible.

Deterministic? Classical logic gates produce outputs **completely determined** by their binary inputs (0 or 1). Each input combination maps to **exactly one output**; no probability, superposition, or ambiguity.

Irreversible? You **cannot uniquely recover inputs** from the output alone. Information is **lost** during computation because multiple input states collapse to the same output.

1.1 Truth Tables

1.1.1 AND Gate

Output is 1 **only if both inputs are 1**.

A	B	**A AND B**
0	0	0
0	1	0
1	0	0
1	1	**1**

1.1.2 OR Gate

Output is 1 **if at least one input is 1**.

A	B	**A OR B**
0	0	0
0	1	**1**
1	0	**1**
1	1	**1**

1.1.3 NOT Gate (Inverter)

Flips the single input.

A	**NOT A**
0	**1**
1	**0**

1.2 Irreversibility Examples

1.2.1 AND Gate Failure Case

Output = 0. What were inputs A and B?

- Could be: (0,0), (0,1), or (1,0)
- **Three possible input states → one output**
- **Information destroyed!**

1.2.2 OR Gate Failure Case

Output = 1. What were inputs?

- Could be: (0,1), (1,0), or (1,1)
- **Again, multiple inputs map to same output.**

Quantum gates

Quantum gates are **always reversible unitary matrices** U satisfying:

$$U^\dagger U = I \quad \text{(every output has unique pre-image)}$$

This means every quantum operation has an **inverse** $U^{-1} = U^\dagger$ that perfectly recovers the original state.

1.3 Core Quantum Gates

1.3.1 CNOT = Reversible XOR

Control qubit unchanged, target flips if control=1.

Control	Target	CNOT Output	Reversible?
$\|0\rangle$	$\|0\rangle$	$\|00\rangle$	✓
$\|0\rangle$	$\|1\rangle$	$\|01\rangle$	✓
$\|1\rangle$	$\|0\rangle$	$\|11\rangle$	✓
$\|1\rangle$	$\|1\rangle$	$\|10\rangle$	✓

1.3.2 Toffoli (CCNOT) = Reversible AND

A	B	C	Toffoli $\|ABC\rangle$	Reversible?
0	0	0	$\|000\rangle$	✓
0	0	1	$\|001\rangle$	✓
0	1	0	$\|010\rangle$	✓
0	1	1	$\|011\rangle$	✓
1	0	0	$\|100\rangle$	✓
1	0	1	$\|101\rangle$	✓
1	1	0	$\|111\rangle$	✓
1	1	1	$\|110\rangle$	✓

1.4 Perfect Reversibility

Apply Toffoli **twice**:

$$|abc\rangle \xrightarrow{\text{Toffoli}} |ab(c \oplus ab)\rangle \xrightarrow{\text{Toffoli}} |abc\rangle$$

Toffoli is its own inverse! $|abc\rangle \to |ab(c \oplus ab)\rangle \to |abc\rangle$

1.5 Operate on Complex Amplitudes

Unlike classical bits, quantum gates act on **superpositions**:

$$\text{CNOT on } \frac{|00\rangle + |11\rangle}{\sqrt{2}} = \frac{|00\rangle + |10\rangle}{\sqrt{2}}$$

$$\text{Toffoli on } \frac{|000\rangle + |111\rangle}{\sqrt{2}} = \frac{|000\rangle + |110\rangle}{\sqrt{2}}$$

1.6 Truth Table Verification

Classical	Irreversible?	Quantum Equivalent
AND	× (loses info)	Toffoli (CCNOT)
XOR	× (loses info)	CNOT

1.7 Key Insight

Classical irreversibility requires **extra bits** for reversibility (Fredkin/Toffoli gates in classical reversible computing). Quantum computing gets this **for free** through unitary evolution.

Classical gates destroy information. Quantum gates preserve it.

1.8 Why This Matters

- **No information loss** → No heat generation from Landauer's principle
- **Superposition preserved** → Massive parallelism
- **Universal decomposition** → Any quantum algorithm implementable

2 Single-Qubit Gates

Single-qubit unitaries rotate or phase-shift states on the Bloch sphere.

Gate	Matrix	Bloch Axis
X	$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$	180° about X
Y	$\begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix}$	180° about Y
Z	$\begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$	180° about Z
H	$\frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$	Maps $\|0\rangle to \|+\rangle$
S	$\begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix}$	90° Z rotation
T	$\begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$	45° Z rotation

Pauli Matrices in Qiskit

```
X = Operator.from_label('X')
Z = Operator.from_label('Z')

print("Pauli-X:\n", X.data)
print("Pauli-Z:\n", Z.data)
```

Bloch Visualization

```
# Visualize Pauli effects from |+>
# Start from |+> and show, on the Bloch sphere, what happens when you apply
    each Pauli gate X, Y, Z to that state.
sv_plus = Statevector.from_label('+')

states = [
    sv_plus,
    sv_plus.evolve(X),
    sv_plus.evolve(Y),
    sv_plus.evolve(Z)
]

titles = ['|+>', 'X|+> = |+>', 'Y|+>=i|->', 'Z|+>=|->']

# Note: In 2.1, plot_bloch_multivector returns a Figure.
# You can display them sequentially:
for state, title in zip(states, titles):
    display(plot_bloch_multivector(state, title=title))
```

Calculation for $X|+\rangle$

$$X|+\rangle = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \cdot 1 + 1 \cdot 1 \\ 1 \cdot 1 + 0 \cdot 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = |+\rangle$$

Calculation for $Y|+\rangle$

$$Y|+\rangle = \begin{pmatrix} 0 & -i \\ i & 0 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 0 \cdot 1 + (-i) \cdot 1 \\ i \cdot 1 + 0 \cdot 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} -i \\ i \end{pmatrix} = -i \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = -i|-\rangle$$

Calculation for $Z|+\rangle$

$$Z|+\rangle = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix} \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \cdot 1 + 0 \cdot 1 \\ 0 \cdot 1 + (-1) \cdot 1 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ -1 \end{pmatrix} = |-\rangle$$

```
# Clifford + T = Universal
gates = ['h', 's', 't']
sv0 = Statevector.from_label('0')

# 1. Plot the initial state
display(plot_bloch_multivector(sv0, title='|0>'))

# 2. Loop through and plot each gate evolution
for gate in gates:
    qc = QuantumCircuit(1)
    # Get the gate method from the circuit (e.g., qc.h) and call it on qubit 0
    getattr(qc, gate)(0)

    # Evolve the statevector with the circuit
    sv_evolved = sv0.evolve(qc)

    # Display the result (removes the need for axes[i])
    display(plot_bloch_multivector(sv_evolved, title=f'Gate: {gate.upper()}'))
```

Hadamard Gate (H) on $|0\rangle$

$$H|0\rangle = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \cdot 1 + 1 \cdot 0 \\ 1 \cdot 1 + (-1) \cdot 0 \end{pmatrix} = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 \\ 1 \end{pmatrix} = |+\rangle$$

Phase Gate (S) on $|0\rangle$

$$S|0\rangle = \begin{pmatrix} 1 & 0 \\ 0 & i \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 1 + i \cdot 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle$$

T Gate (T) on $|0\rangle$

$$T|0\rangle = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix} \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 1 \cdot 1 + 0 \cdot 0 \\ 0 \cdot 1 + e^{i\pi/4} \cdot 0 \end{pmatrix} = \begin{pmatrix} 1 \\ 0 \end{pmatrix} = |0\rangle$$

3 Multi-Qubit Gates

3.1 CNOT and Bell State Formation

```
qc_cnot = QuantumCircuit(2)
qc_cnot.h(0)  # Superposition control
qc_cnot.cx(0, 1)
sv_cnot = Statevector.from_label('00').evolve(qc_cnot)
print("Bell state:", sv_cnot)
plot_state_qsphere(sv_cnot.data)
```

Result: The circuit produces $\frac{|00\rangle+|11\rangle}{\sqrt{2}}$: the $|\Phi^+\rangle$ Bell state.

3.2 Toffoli (CCNOT)

The **Toffoli gate** (CCNOT) is a 3-qubit gate with **two control qubits** and **one target qubit**. It applies an X (NOT) operation to the target **only if both controls are** $|1\rangle$.

$$|ABC\rangle \xrightarrow{\text{CCNOT}} |AB(C \oplus (A \cdot B))\rangle$$

Truth Table

A	B	C	$\|ABC\rangle$	Action
0	0	0	$\|000\rangle$	No change
0	0	1	$\|001\rangle$	No change
0	1	0	$\|010\rangle$	No change
0	1	1	$\|011\rangle$	No change
1	0	0	$\|100\rangle$	No change
1	0	1	$\|101\rangle$	No change
1	1	0	$\|110\rangle$	$\rightarrow \|111\rangle$
1	1	1	$\|111\rangle$	$\rightarrow \|110\rangle$

8×8 Unitary Matrix

In computational basis $\{|000\rangle, |001\rangle, \ldots, |111\rangle\}$:

$$\text{CCNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

Only **swaps rows/columns** $6 \leftrightarrow 7$ (i.e., $|110\rangle \leftrightarrow |111\rangle$).

Matrix Action Examples

Classical Case: $|110\rangle$

$$\text{CCNOT} \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = |111\rangle$$

Superposition Case

$$\text{CCNOT} \frac{|000\rangle + |110\rangle}{\sqrt{2}} = \frac{|000\rangle + |111\rangle}{\sqrt{2}}$$

Block-Diagonal Form

$$\text{CCNOT} = \begin{pmatrix} I_6 & 0 \\ 0 & X \end{pmatrix}$$

where I_6 is 6×6 identity, X is Pauli-X matrix.

Why "Reversible AND" ?

- **AND result** stored in target: $C' = C \oplus (A \cdot B)$
- **Inputs preserved** in first two qubits
- **Apply twice** $\rightarrow$ back to original state

$$|ABC\rangle \xrightarrow{\text{CCNOT}} |AB(C \oplus AB)\rangle \xrightarrow{\text{CCNOT}} |ABC\rangle$$

Symbol Guide

Symbol	Meaning
A, B, C	Qubit values (0 or 1)
$\|ABC\rangle$	3-qubit state
$\oplus$	XOR: $0 \oplus 0 = 0$, $0 \oplus 1 = 1$, $1 \oplus 0 = 1$, $1 \oplus 1 = 0$
AB	AND: $A \cdot B$ (1 only if both 1)
$C \oplus AB$	"C XOR (A AND B)"

Step-by-Step: $|110\rangle$

1. **Inputs:** $A = 1$, $B = 1$, $C = 0$
2. **AND:** $AB = 1 \cdot 1 = 1$
3. **XOR:** $C \oplus AB = 0 \oplus 1 = 1$
4. **New C:** $C' = 1$
5. **Output:** $|11\,1\rangle$

$$|110\rangle \rightarrow |11(0 \oplus (1 \cdot 1))\rangle = |111\rangle$$

Why Reversible? Second application:

1. **Inputs:** $A = 1$, $B = 1$, $C = 1$
2. **AND:** $AB = 1 \cdot 1 = 1$
3. **XOR:** $C \oplus AB = 1 \oplus 1 = 0$
4. **New C:** $C' = 0$
5. **Output:** $|110\rangle$

Truth Table

A	B	C	New C $= C \oplus (AB)$
1	1	0	$0 \oplus 1 = 1$
1	1	1	$1 \oplus 1 = 0$
0	–	–	$C \oplus 0 = C$ (unchanged)

"When both controls = 1, target FLIPS. Otherwise unchanged."

4 Universal Gates & Decomposition

4.1 SWAP as 3×CNOT

The two-qubit **SWAP** gate exchanges the states of two qubits:

$$|a, b\rangle \;\mapsto\; |b, a\rangle.$$

In circuit form, it can be decomposed into three CNOT gates:

$$\mathrm{SWAP}(q_0, q_1) = \mathrm{CNOT}_{q_0 \to q_1}\ \mathrm{CNOT}_{q_1 \to q_0}\ \mathrm{CNOT}_{q_0 \to q_1}.$$

Read this as: "apply a CNOT with q_0 as control and q_1 as target, then a CNOT with q_1 as control and q_0 as target, then the first CNOT again."

Truth Tables

SWAP Gate (abstract definition)

In the computational basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$,

Input	SWAP Output
$\|00\rangle$	$\|00\rangle$
$\|01\rangle$	$\|10\rangle$
$\|10\rangle$	$\|01\rangle$
$\|11\rangle$	$\|11\rangle$

CNOT

A CNOT with control q_0 and target q_1 acts as:

$$|a, b\rangle \mapsto |a,\ b \oplus a\rangle,$$

where $\oplus$ is XOR.

Truth table for $\mathrm{CNOT}_{0\to1}$:

Input	Output
$\|00\rangle$	$\|00\rangle$
$\|01\rangle$	$\|01\rangle$
$\|10\rangle$	$\|11\rangle$
$\|11\rangle$	$\|10\rangle$

Similarly, $\mathrm{CNOT}_{1\to0}$ uses q_1 as control and flips q_0.

Matrix Formalism

SWAP Matrix

In basis $\{|00\rangle, |01\rangle, |10\rangle, |11\rangle\}$, the SWAP gate is

$$\mathrm{SWAP} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

This matrix leaves $|00\rangle$ and $|11\rangle$ fixed, and swaps $|01\rangle \leftrightarrow |10\rangle$.

CNOT Matrices

For control q_0, target q_1:

$$\mathrm{CNOT}_{0\to1} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}.$$

For control q_1, target q_0:

$$\mathrm{CNOT}_{1\to0} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{pmatrix}.$$

Composition: 3×CNOT = SWAP

Matrix composition corresponds to multiplying the matrices (rightmost gate acts first):

$$\mathrm{SWAP} = \mathrm{CNOT}_{0\to1}\ \mathrm{CNOT}_{1\to0}\ \mathrm{CNOT}_{0\to1}.$$

If you multiply these 4×4 matrices, the result is exactly the SWAP matrix above.

Example Calculation on a Basis State

Take input $|01\rangle$ (i.e. $q_0 = 0, q_1 = 1$) and apply the three CNOTs step by step.

Step 1: First $\mathrm{CNOT}_{0\to1}$

Control is 0, so target does not flip:

$$|01\rangle \xrightarrow{\mathrm{CNOT}_{0\to1}} |01\rangle.$$

Step 2: $\mathrm{CNOT}_{1\to0}$

Now control is $q_1 = 1$, so q_0 flips:

$$|01\rangle \xrightarrow{\mathrm{CNOT}_{1\to0}} |11\rangle.$$

Step 3: Second $\mathrm{CNOT}_{0\to1}$

Control is $q_0 = 1$, so q_1 flips:

$$|11\rangle \xrightarrow{\mathrm{CNOT}_{0\to1}} |10\rangle.$$

Overall:

$$|01\rangle \longrightarrow |10\rangle,$$

which matches the SWAP action.

A **universal gate set** can create **any possible quantum circuit** through combinations.

5 Two Universal Sets

Universal Set	What it can do
{H, T, CNOT}	**Any** quantum circuit
{H, S, CNOT}	Any **Clifford** circuit

"{H, T, CNOT} = universal" means any quantum operation can be built using **only** these 3 gates.

The Clifford circuit: A circuit made **only** from these 6 gates:

- Single-qubit: H, S, X, Y, Z (Pauli gates)
- Two-qubit: CNOT

Clifford gates = "easy gates" that preserve the stabilizer structure

Why T Gate Makes It Universal

The **T gate** (45° phase) breaks Clifford symmetry:

$$T = \begin{pmatrix} 1 & 0 \\ 0 & e^{i\pi/4} \end{pmatrix}$$

T creates states that cannot be simulated efficiently by Clifford circuits alone.

What Can Clifford Circuits Do?

1. Create Bell states: $\frac{|00\rangle+|11\rangle}{\sqrt{2}}$
2. Error correction (stabilizer codes)
3. NOT universal quantum computing

Why This Distinction Matters

- **Clifford**: Efficiently classically simulable
- **Universal**: Quantum supremacy requires T gates
- Real quantum hardware: T gates are ∼100x slower than Clifford

Quantum Circuit Optimization

It shows how Qiskit can take a "messy" circuit and simplify it into the mathematically shortest version possible.

```
from qiskit import QuantumCircuit, transpile

# Create a "messy" circuit with 4 gates:
qc_complex = QuantumCircuit(2)
qc_complex.h(0)
qc_complex.cx(0,1)
qc_complex.h(0)
qc_complex.cx(0,1)

print("Before optimization:", len(qc_complex.data))

# Fix: Provide a basis_gates set (like IBM's standard) so level 3 knows how to
    collapse gates
```

```
# Common basis: ['u1', 'u2', 'u3', 'cx'] or ['id', 'rz', 'sx', 'x', 'cx']
optimized = transpile(qc_complex, optimization_level=3, basis_gates=['u3', 'cx'])
#optimization_level=3: This is the highest setting. Qiskit looks for patterns it can shrink.
#Translate my circuit using only the universal U3 (rotation) and CX (CNOT) gates.

#On real quantum computers, every gate adds noise and error.
#Optimization level 3 is used to make the circuit as "short"
#as possible to get the most accurate results before the qubits decohere (die).

print("After:", len(optimized.data))
optimized.draw('mpl')
```

6 Exercises

1. Build a controlled-H gate using H, CNOT, and Z.
2. Create a 3-qubit GHZ state: $\frac{|000\rangle+|111\rangle}{\sqrt{2}}$.
3. Decompose Toffoli into {H, T, CNOT}.

Hints: Mathematical Form of the Controlled-Hadamard (CH) Gate

The controlled-Hadamard gate acts on two qubits. It applies the Hadamard operator (H) to the target qubit if and only if the control qubit is in the state $|1\rangle$. This "conditional" logic is expressed using the projector-operator notation:

$$CH = |0\rangle\langle 0| \otimes I + |1\rangle\langle 1| \otimes H$$

Breakdown of Components

- **Control Projectors:** $|0\rangle\langle 0|$ and $|1\rangle\langle 1|$ act on the first qubit. They "select" the components of the state where the control qubit is 0 or 1, respectively.
  - $|0\rangle\langle 0| = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}$: This "selects" the component of the state where the first qubit is 0.
  - $|1\rangle\langle 1| = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}$: This "selects" the component of the state where the first qubit is 1.
- **Target Operations:** I (Identity) and H (Hadamard) act on the second qubit. The tensor product $\otimes$ symbol glues the "control" action to the "target" action.
  - $|0\rangle\langle 0| \otimes I$: "If the first qubit is 0, apply Identity to the second."
  - $|1\rangle\langle 1| \otimes H$: "If the first qubit is 1, apply Hadamard to the second."
- **The Sum:** The $+$ sign combines these two conditional branches into a single linear operator.

Matrix Representation

By substituting the matrix forms of the projectors and operators:

$$|0\rangle\langle 0| = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix}, \quad |1\rangle\langle 1| = \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix}, \quad I = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}, \quad H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

The tensor products yield:

$$CH = \begin{pmatrix} 1 & 0 \\ 0 & 0 \end{pmatrix} \otimes \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \begin{pmatrix} 0 & 0 \\ 0 & 1 \end{pmatrix} \otimes \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

Resulting in the block-diagonal matrix:

$$CH = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{1}{\sqrt{2}} & \frac{1}{\sqrt{2}} \\ 0 & 0 & \frac{1}{\sqrt{2}} & -\frac{1}{\sqrt{2}} \end{pmatrix}$$

Key Takeaways:

- Pauli + H form the single-qubit basis.
- CNOT introduces entanglement.
- {H, T, CNOT} = universal.
- Decomposition + optimization map logical to hardware-efficient circuits.